PhishLens: A Lightweight, Explainable URL Phishing Detection System

A Time-, Source-, Provenance- and Evasion-Aware Benchmark with Drift-Aware Selective Hardening

Rajarajeswari V
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India
rajarajeswarivisu@gmail.com

Ms. MeenaPriyadharsini A
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India
meenapa@hindustanuniv.ac.in

Arivu Nithi M
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India
arivunithi6385@gmail.com

Kevin Abishek K
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India
kingkevin.72581@gmail.com

Jeff Adrain G
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India
Jeffadrain262@gmail.com

***Abstract*—We introduce PhishLens, a lightweight, explainable URL phishing detector, built on PhishDriftBench (a four-axis benchmark spanning time, source, evasion and deployment prevalence) and DASH, a drift-aware mitigation. Published detectors routinely report 97–99.5% accuracy, but an audit of ten recent papers (2024–2026) finds none evaluates under a temporal split, reports precision at a realistic deployment base rate, or tests its detector against cataloged evasion, and only one performs genuine cross-source transfer. Re-evaluating five architectures under PhishDriftBench, the reported–deployed gap proves real but uneven: temporal degradation was undetectable (all five within 0.003 AUC of ceiling); cross-source transfer cost up to 8.9 AUC points and reordered the leaderboard; rule-based evasion mostly failed, while generative evasion cost every model 4.5–5.4 recall points uniformly. Sharpest of all is prevalence correction: at a realistic 1-in-10,000 rate, precision for three of five baselines falls to 4.8–11.2% despite statistically indistinguishable 98–99% balanced accuracy: accuracy is shown to be actively uninformative about deployment false-alarm rate. A provenance audit (285 live lookups) finds no accuracy gain from lookup features (-0.51 AUC), and DASH's null result on this corpus traces to an easy negative class, not the mitigation: a harder legitimate class produced real drift alarms at a cost of 20 labels.**

We then build PhishLens (B6), adding one fix per weakness and testing each in isolation: cleaning alone does not help precision, adversarial augmentation trades recall for a higher false-positive rate, and a two-stage cascade (a real classifier, not a heuristic) recovers that cost and more, cutting false positives sixfold and raising precision at $\pi{=}10^{-4}$ from 1.8% to 9.95%, above three of the five original baselines. B7's unsupervised novelty gate shows a naive version's recall gain collapses precision to 0.20%, while an orthogonal revision recovers 75% of the benefit at a third of the cost. PhishLens exposes every verdict through exact TreeSHAP attribution, brand-jacking reason codes and a calibrated abstention band, implemented in a working tool. Auditing this layer shows 67.15% of its attribution weight rests on features independently flagged drift-brittle, and traces a real misclassification (a legitimate Wikipedia URL scored 0.9999 phishing) to exactly that reliance: an explanation faithful today carries no guarantee of remaining faithful as traffic drifts.

***Index Terms*—phishing detection, URL classification, explainable AI, SHAP, concept drift, dataset shift, evaluation methodology, adversarial robustness, machine learning security, data leakage, benchmark, lightweight detection**

I. Introduction

Phishing remains among the most economically damaging classes of cybercrime, and URL-based detection is attractive because it operates before any content is fetched or rendered, imposing negligible latency and no rendering infrastructure. A large body of work reports this problem close to solved: accuracies of 98.29% [1], 99.45% [2], 99.48% [3] and 99.35% [4] have all appeared in the last two years.

These numbers are not fraudulent, but answer a question that is not the deployment question. Almost without exception they come from randomly partitioning a single corpus collected over one time window, balanced at roughly equal class prevalence, with no adversary in the loop. Each of those four choices is individually defensible and collectively misleading:

1) **Random splits leak time.** Phishing URLs arrive in campaigns (one kit, one registrar burst, thousands of near-identical strings), so a random split grades the model on near-duplicates of what it memorised.
2) **Single-source evaluation hides source-specific artifacts.** A feature can carry *opposite* meaning in different corpora: Misiek and Hyla [5] report HTTPS presence characterising 100% of legitimate URLs in PhiUSIIL but only 59.2% in their own corpus.
3) **Balanced test sets hide the base-rate problem.** A 99% accuracy, 1% false-positive detector deployed

at 1:10^4 prevalence yields precision near 1%, roughly ninety-nine false alarms per true detection. Reported accuracy carries none of this information.

4) **Adversaries are catalogued but never applied.** Li *et al.* [6] systematically review cloaking and evasion techniques; no method paper in our corpus evaluates its detector against them.

Exactly one paper in our corpus measures the consequence. Misiek and Hyla [5] report 97.80% under single-dataset evaluation and 77.84% averaged across three sources, a 19.96 percentage-point collapse that they correctly attribute to dataset shift, "a factor often overlooked in existing literature."

A. Contributions

- **C1.** An evaluation-protocol audit of ten recent phishing detection papers, eight of them method studies from 2024–2026 (Table I), establishing that the protocol gap is current rather than historical.
- **C2. PhishDriftBench**: a four-axis protocol evaluating detectors across time (Axis T), across sources (Axis S), against rule-based and generative evasion (Axis E), and at deployment-realistic class priors.
- **C3.** A **feature-provenance and lookup-dependency audit** that partitions features by whether they require a network lookup, and quantifies the accuracy attributable to retrospective-lookup artifacts.
- **C4.** A **drift-stability scoring** method that identifies drift-brittle features before deployment.
- **C5. DASH**, a lightweight drift-aware mitigation combining stability-weighted features, unsupervised drift detection, a bounded active-labelling budget and a calibrated abstention band.
- **C6. B6**, a model built incrementally so each component's contribution is measured, not assumed: data cleaning, stability-weighted training, adversarial generative augmentation, and a two-stage cascade consulted only where Stage 1 is not confident.
- **C7. B7**, an unsupervised novelty gate for zero-day phishing: a naive version's recall benefit does not survive contact with prevalence-corrected precision, but diagnosing why motivates a revision (an orthogonal, character-sequence signal) that recovers most of the benefit at a fraction of the cost.
- **C8. PhishLens**: we name the deployed detector (B6's v3 cascade) and add an explainability layer (exact TreeSHAP attribution, brand-jacking reason codes, a calibrated UNCERTAIN verdict) implemented in a working tool. Auditing it against C4's scores shows 67.15% of its attribution weight rests on features independently flagged drift-brittle, and we trace a real misclassification (a legitimate Wikipedia URL scored 0.9999) to exactly that reliance.

Throughout, comparative claims are backed by non-parametric bootstrap 95% confidence intervals over the relevant test set (1,000 resamples), added in this revision so headline numbers carry an explicit uncertainty estimate rather than a bare point value (Tables III, IV, VII).

II. Related Work and Protocol Audit

A. Feature-engineering and classical ML approaches

Kustiawan and Ghauth compare URL-, HTML- and derived-feature sets across ten classifiers [2] and later propose PhishOFE [3], an optimised feature-engineering framework avoiding third-party features, reaching 99.48% with CatBoost. Goenka *et al.* [4] propose a two-layer model where a domain-squatting/obfuscation screen precedes a lexical classifier (99.35% at 12.49 ms), but only 89.17% on an older corpus, which they attribute, correctly, to 2025's top targeted brands differing from 2018's: a direct observation of concept drift, measured but untreated.

B. Deep and hybrid architectures

Remya *et al.* [1] apply a residual pipeline of convolutional and inverted-residual blocks feeding an MLP over lexical, sentiment and domain-age features, later proposing BGL-PhishNet [7] (BERT + GNN + LightGBM, 97.3% on 549,346 URLs). He *et al.* [8] introduce a TextRank feature-extension library feeding a BERT–CNN classifier. Vulfin *et al.* [9] fuse four modalities (CatBoost, a character CNN, CodeBERT, EfficientNet-B7 over screenshots) with SHAP/Grad-CAM explanation, and are among the few to quantify temporal decay (8–15% degradation).

C. AI-generated phishing URLs, and reviews of the field

Early work assumed a narrow generative model: DeepPhish [10], a recurrent network fitted to one threat actor's URLs, was for years the only documented URL generator, and detectors were evaluated against it accordingly, a premise that does not hold in 2026, when general-purpose language models produce diverse candidate URLs at negligible cost. **Whether a detector validated against a narrow, historical generator retains its recall against a contemporary one is, to our knowledge, untested for URL-only detection.** Axis E2 tests this directly. Li *et al.* [6] systematically review server- and client-side cloaking, documenting how phishing infrastructure fingerprints requesters to serve benign content selectively to analysis systems; Kavya and Sumathi [11] survey the field from list-based through deep and graph-based methods, naming dataset diversity, adversarial robustness and interpretability as unfilled gaps.

D. Protocol audit

We extracted the full text of every paper in the corpus and searched systematically for evaluation-protocol terms (Table I). Unlike every other table here, its cells are established facts about the literature, not experimental measurements.

TABLE I
EVALUATION-PROTOCOL AUDIT OF THE SURVEYED CORPUS. ✓ = PRESENT, – = ABSENT. MENTIONS IN PROSE WITHOUT AN ACCOMPANYING EXPERIMENT ARE COUNTED ABSENT.

Work	Year	Temp.	X-src	Prev.	Evas.
ResMLP [1]	2024	–	–	–	–
Feature Ext. [8]	2024	–	–	–	–
Feat. Eng. [2]	2025	–	–	–	–
PhishOFE [3]	2025	–	–	–	–
Layered [4]	2025	–	†	–	–
BGL-PhishNet [7]	2025	–	–	–	–
Multimodal XAI [9]	2026	–	–	–	–
X-PHIDE [5]	2026	–	✓	–	–
This work	–	✓	✓	✓	✓

† = evaluated on multiple datasets but *retrained per dataset*, which measures dataset difficulty rather than transfer. Reviews [6], [11] are excluded as they report no primary evaluation.

The audit yields the finding that motivates this paper: across all eight method papers in the corpus, published between 2024 and 2026 (that is, among the most recent work in the field, not a historical sample), temporal evaluation, prevalence-corrected reporting and self-directed evasion testing are each performed by *zero* papers, and genuine cross-source transfer by one.

III. Threat Model and Problem Formulation

A. Deployment setting

We consider a detector $f_\theta : \mathcal{U} \to [0,1]$ mapping a URL string $u \in \mathcal{U}$ to a phishing score, deployed inline in a browser extension or mail gateway. Three properties distinguish deployment from the standard experimental setting:

- **Temporal ordering.** The detector is trained on URLs observed up to time T and must classify URLs first observed after T. The joint distribution $P_t(u, y)$ is non-stationary.
- **Severe class imbalance.** Deployment prevalence $\pi = P(y{=}1)$ lies in the range 10^{-2} to 10^{-4}, not 0.5.
- **Adaptive adversary.** The attacker observes deployed detectors and applies label-preserving transformations τ such that $\tau(u)$ remains an effective phishing URL while $f_\theta(\tau(u))$ falls below threshold.

B. Adversary capabilities

We assume the adversary can (i) apply lexical transformations to a URL without altering its function, (ii) generate novel URL strings using a contemporary generative model, and (iii) control any *lookup channel* the detector depends on, for example serving a benign redirect target to a detector that expands shortened URLs, consistent with the cloaking behaviour documented in [6]. We do *not* assume white-box gradient access to the deployed model.

C. The reported–deployed gap

Let A_{rand} denote accuracy under a random split of a single corpus, and A_{deploy} denote accuracy under temporal, cross-source and adversarial conditions at deployment prevalence. We define the *reported–deployed gap*

$$\Delta = A_{\text{rand}} - A_{\text{deploy}}, \tag{1}$$

and take the estimation of Δ, its decomposition into temporal, cross-source, provenance and adversarial components, and its mitigation, as the object of this work.

IV. PhishDriftBench

A. Axis T: temporal generalisation

Given a corpus with first-observation timestamps, we select a cut point T, train on $\{u : t(u) \le T\}$, and evaluate on disjoint windows $(T, T{+}1]$, $(T, T{+}3]$, $(T, T{+}6]$ and $(T, T{+}12]$ months. The output is a *decay curve* $A(\delta)$ rather than a scalar. Where a source lacks per-URL timestamps we fall back to dated dataset *releases* as a coarse time axis, exploiting the 2018–2026 span of the corpora in this study.

a) Asymmetric timestamp granularity (methodological caveat).: Only PhishTank carries genuine per-URL timestamps among the acquired corpora; PhiUSIIL and Tranco carry a single dated-release/snapshot stamp and cannot be temporally split. Table III therefore varies only the phishing side by real date, holding the legitimate side to a single fixed Tranco sample. This isolates decay in *recognising new phishing* under a stationary-negative-class assumption, not joint drift in the legitimate population. Separately, PhishTank's bulk file is survivorship-filtered to URLs verified *currently* online, not a stable historical archive. Both caveats are revisited in Limitations.

B. Axis S: cross-source generalisation

For sources $S_1, \ldots, S_n$ we evaluate all ordered pairs $(S_i \to S_j)$, $i \neq j$, plus leave-one-source-out. Crucially, models are trained *once* and transferred, in contrast to [4], which retrains per dataset.

C. Axis E: evasion

a) E1, rule-based: We apply label-preserving transformations drawn from the cloaking and obfuscation taxonomy of [6]: homoglyph and IDN substitution, subdomain padding, path padding, percent-encoding, shortener wrapping, TLD substitution and hyphenated brand insertion. We report recall degradation per transformation.

b) E2, generative: We evaluate against URL strings produced by a contemporary generative model, and compare directly against the DeepPhish-era [10] condition under which early AI-generated-URL detection claims were validated. The research question is stated precisely: *does a detector validated against a 2018-generation URL generator retain its recall against a 2024+ generation generator?*

D. Prevalence correction

For each operating point we report precision, false-positive rate and expected alerts per 10^6 URLs at $\pi \in \{10^{-2}, 10^{-3}, 10^{-4}\}$. Given a threshold τ with true-positive rate $\mathrm{TPR}(\tau)$ and false-positive rate $\mathrm{FPR}(\tau)$ measured on a balanced test set, precision at prevalence π is

$$\mathrm{Prec}_\pi(\tau) = \frac{\pi \cdot \mathrm{TPR}(\tau)}{\pi \cdot \mathrm{TPR}(\tau) + (1-\pi) \cdot \mathrm{FPR}(\tau)}. \quad (2)$$

V. Feature-Provenance and Lookup-Dependency Audit

A. Provenance taxonomy

We classify every feature appearing in the surveyed method papers into three provenance classes:

- **P1, static-lexical.** Computable from the URL string alone, with no network access: length, token counts, entropy, character-class ratios, TLD identity, presence of suspicious tokens.
- **P2, lookup-at-inference.** Requiring a live network operation: WHOIS registration date and expiry, DNS records, SSL certificate presence, domain age, hosting country, and shortened-URL expansion.
- **P3, third-party reputation.** Requiring an external ranking or index: Alexa/Tranco rank, PageRank, search-engine indexing status.

B. The retrospective-lookup artifact

P2 features carry a failure mode not, to our knowledge, named in this literature. When a historical corpus is annotated with P2 features by performing lookups *at write time* rather than at collection time, the resulting values encode domain *mortality* rather than phishing semantics: phishing domains are typically taken down within days, legitimate domains persist for years, so a retrospective query fails for most phishing entries and succeeds for most legitimate ones, and any classifier can exploit this without learning anything about phishing. The hazard is concrete: BGL-PhishNet [7] reports WHOIS/DNS/SSL and host-based features over a 549,346-URL corpus, summarised by a single aggregate "average domain age" with no per-class breakdown. We therefore measure, rather than assume, the magnitude of this effect.

C. Audit experiments

1) **Ablation.** Retrain each baseline with P2 and P3 features removed; report Δaccuracy.
2) **Provenance-timing contrast.** Where a corpus permits it, compare P2 features resolved at collection time against the same features resolved today; report the accuracy difference and the per-class lookup-failure rate.
3) **Lookup-channel adversary.** Measure recall when the lookup channel is (a) unavailable and (b) adversarially controlled, per the cloaking model of [6].

D. Duplicate and near-duplicate leakage probe

Independently of provenance, we quantify exact and near-duplicate URLs straddling the train/test boundary in every corpus used, since campaign structure makes such duplicates likely under random and k-fold partitioning. We report duplicate rate and accuracy before and after deduplication for each dataset, and recommend that this figure be reported as standard.

VI. Drift-Stability Feature Scoring

For feature j we combine distributional stability across time windows and across sources with the variance of its permutation importance. Let $\mathrm{PSI}_t(j)$ and $\mathrm{PSI}_s(j)$ denote the Population Stability Index of feature j computed across consecutive temporal windows and across source pairs respectively, and let $\sigma^2_{\mathrm{imp}}(j)$ denote the variance of its permutation importance across those partitions. We define the stability score

$$\mathrm{Stab}(j) = \Big[\alpha\,\mathrm{PSI}_t(j) + \beta\,\mathrm{PSI}_s(j) + \gamma\,\sigma^2_{\mathrm{imp}}(j)\Big]^{-1}, \quad (3)$$

with α, β, γ fixed on a validation split and never on test data. Features are partitioned into *drift-stable* and *drift-brittle* at a threshold selected on validation data only.

A concrete validation target exists: the HTTPS inversion between PhiUSIIL and the custom corpus of [5] should be flagged as drift-brittle by Eq. (3) *without* access to the second corpus's labels. Whether it is flagged is reported in Section X.

VII. DASH: Drift-Aware Selective Hardening

DASH is deliberately lightweight, so that it remains deployable at the latency budget URL-only detection exists to satisfy. It has four components:

1) **Stability-weighted training.** Train on features weighted by $\mathrm{Stab}(j)$ from Eq. (3), down-weighting drift-brittle features.
2) **Unsupervised drift detection.** Run ADWIN (or DDM) over the stream of prediction confidences. This requires *no labels*, which is the binding constraint in deployment.
3) **Bounded active update.** On a drift alarm, select the most uncertain $b\%$ of the current window ($b \approx 2$), obtain labels for that subset only, and warm-restart or partially refit.
4) **Calibrated abstention.** Define a confidence band in which the model returns UNCERTAIN and escalates to a heavier check rather than guessing. We report a coverage–risk curve rather than a single accuracy.

The claim under test is that DASH recovers a substantial fraction of temporal and cross-source degradation at a small fraction of the labelling cost of full retraining. It is stated as a hypothesis and evaluated in Section X.

VIII. PhishLens (B6): A Proposed Model Addressing the Measured Weaknesses

A. System architecture

Fig. 1 places PhishLens within the full system this paper builds and evaluates it against, end to end: three real corpora (PhishTank, PhiUSIIL, Tranco; Sec. IX-A, Table II) feed the PhishDriftBench protocol, which evaluates five re-implemented baselines and the three proposed contributions — PhishLens (B6), DASH and B7 — built on top of them. PhishLens's own internal structure — Stage 1's lexical classifier, the confidence gate, Stage 2, and the explainability layer built on top of both — is described in detail in Sec. VIII and Sec. VIII-B; Fig. 1 shows where that detector sits relative to the data it was built from, the baselines it is compared against, and the results it is judged by, rather than repeating that internal detail here.

Sections X and XI establish four concrete weaknesses in B1–B5. This section proposes one model, built incrementally, that targets each weakness directly rather than proposing architecture changes disconnected from what was actually measured. Each version below adds exactly one component on top of the last, so that its individual contribution can be isolated experimentally (Sec. X-I).

v0 (clean data). Targets two quantified data artifacts: 19.4% near-duplication in PhishTank (Table IX) and homoglyph substitution's unpunished effect on B3 (Table V). v0 applies LSH near-duplicate removal and Unicode confusable normalization (mapping each non-ASCII character to its true-Latin visual confusable) as a *mandatory* preprocessing step before training an otherwise-unchanged gradient-boosted lexical classifier.

v1 (+ stability-weighted training). Features are weighted by Stab(j) (Eq. (3)) via XGBoost's native `feature_weights`, biasing column subsampling away from drift-brittle features (Sec. VI). This is DASH's first component (Sec. VI), applied here as a default training recipe rather than only inside the DASH loop.

v2 (+ adversarial generative augmentation). The training set is augmented with 3,000 synthetic URLs (1,500 each) from the two character-Markov generators used for Axis E2, labelled phishing. This targets the one finding that hit every baseline uniformly: recall loss against a contemporary-style generator (Table VI).

v3 (+ two-stage cascade). v2 becomes a high-recall Stage 1 filter: URLs scoring below 0.1 are dispositioned negative immediately; the rest pass to Stage 2, a real B5 (BERT embeddings + LightGBM) fit once on the same training set, whose score becomes final for routed URLs. This targets the largest effect measured in this work: precision collapse at deployment prevalence (Table VII). An initial Stage 2 used a hand-rolled nearest-BERT-centroid heuristic instead of a trained classifier; it collapsed recall from 0.99 to 0.757 with no precision gain. It is reported in Sec. X-I as evidence about cascade *design* (Stage 2 must be an actually-trained classifier), not against the cascade *idea*, which is why v3 uses the real, already-validated B5 pipeline instead.

B. PhishLens: naming the deployed system, and its explainability layer

We call the resulting detector **PhishLens**: the v3 cascade above (Stage 1: 33 P1 lexical features + XGBoost; Stage 2: DistilBERT embeddings + LightGBM for the subset Stage 1 does not confidently clear), instrumented with three additions that make its decisions inspectable rather than opaque, none of which changes the classification numbers in Sec. X-I, since they sit downstream of the score, not inside it.

1) **Per-URL feature attribution.** Every score carries an *exact* TreeSHAP explanation, computed with XGBoost's native `pred_contribs=True` for Stage 1 (the same values the `shap` package's `TreeExplainer` would produce, with no approximation), reporting which features pushed the score toward PHISHING or LEGITIMATE, and by how much.
2) **Brand-jacking reason codes.** B2's domain-squatting screen (Sec. X) is exposed as a human-readable reason (e.g. "looks like a misspelling of a known brand domain") rather than folded silently into the score.
3) **Calibrated abstention.** DASH's abstention band (Sec. VII, component 4) is surfaced as a third verdict, UNCERTAIN, with a stated reason (score inside the calibrated band, or a link-shortener whose destination this system deliberately does not resolve). It refuses to guess past the point the model has evidence for, rather than forcing a binary answer.

All three are implemented in a working tool (CLI and web), not only described. The quantitative results in Sec. X-L use a 33-feature XGBoost classifier matching Stage 1, fit independently on a similarly-constructed sample (weights uniform for this fit, not Table XI's exact weights); the claim under test is the ranking of feature importance, not one fitted model's precise score.

C. B7: an unsupervised novelty gate for zero-day phishing

B1–B6 share a structural ceiling no amount of retraining removes: every one is purely supervised, recognising only phishing *patterns present in its training data*, the direct cause of Axis E2's 4.5–5.4 recall-point loss against an unseen contemporary generator (Table VI). B7 proposes a structurally different mechanism: an Isolation Forest [12]-style anomaly detector fit *only* on legitimate URLs, flagging anything far from the "normal-legitimate" manifold regardless of the supervised cascade's verdict, the one case a purely supervised model cannot handle by construction.

Three combination rules are tested against B6's v2+cascade as the supervised component: **OR-gate.** Flag

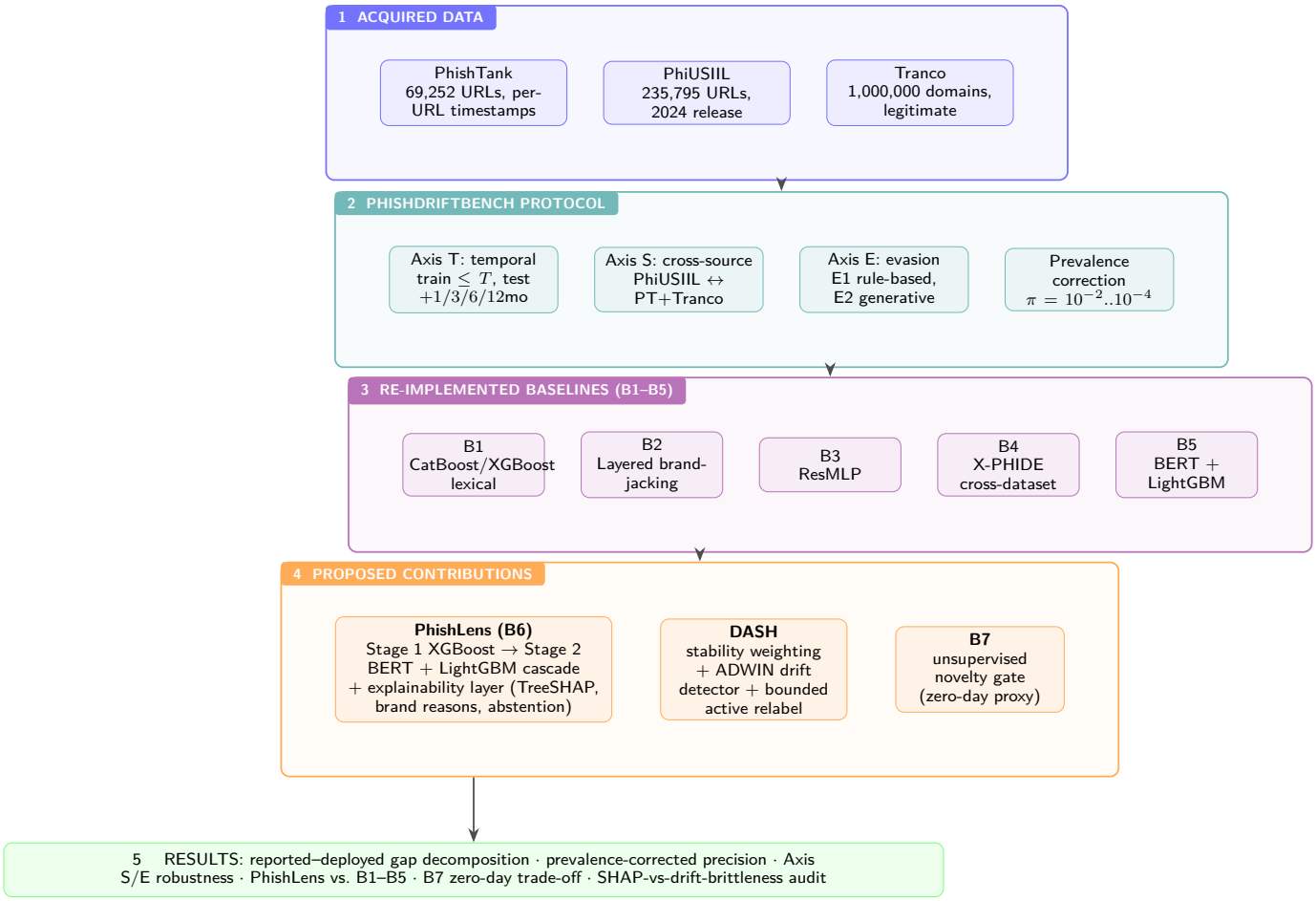


Fig. 1. PhishLens's place in the full system: the three acquired corpora feed the PhishDriftBench protocol, which evaluates both the five re-implemented baselines and the three proposed contributions this paper builds on top of them. PhishLens's own internal cascade and explainability layer are detailed in Sec. VIII and Sec. VIII-B.

phishing if the cascade says so *or* the novelty score exceeds a threshold calibrated on the 95th percentile of a legitimate *validation* split.

Gated refinement. Let novelty escalate only URLs the cascade is already uncertain about (score $\in [0.15, 0.5)$), rather than blanket-flagging every novel-looking URL.

AND-gate (revision). Both above operate on one novelty signal, the Isolation Forest on the same 33 structural features the supervised cascade uses. Sec. X-J traces the OR-gate's false-positive cost to exactly this overlap: "structurally unusual" and "generated-style phishing text" share a large fraction of ordinary legitimate URLs. The AND-gate adds a second signal on a genuinely different feature space, namely character *sequences*: an order-4 character Markov model (add-k-smoothed, $k = 0.5$) fit only on legitimate URL text, scoring average per-character surprisal. A structurally-unusual but textually-ordinary legitimate URL should score low here even where the structural gate scores it high; Axis E2's Markov-generated text is fit on *phishing* strings, so it should score high on both signals. Escalation requires *both* to agree (same calibration), which is strictly more conservative than the OR-gate by construction, since it can only suppress false positives the OR-gate would raise, never add new ones.

Axis E2's contemporary-generator URLs serve as the zero-day proxy: they are, by construction, phishing text no component of B6 has trained on.

IX. Experimental Setup

A. Datasets

Dataset sizes stated without placeholders are as reported by the originating papers; all sizes are re-verified after deduplication and the verified counts reported in Section X.

B. Baselines

We re-implement five architectures spanning the design space of the surveyed corpus:

1) **B1: Gradient-boosted lexical.** CatBoost/XGBoost over engineered lexical features, following [2], [3].

TABLE II

CORPORA USED. TIMESTAMP GRANULARITY DETERMINES ELIGIBILITY FOR AXIS T. ROWS MARKED * ARE ACQUIRED AND VERIFIED BY THIS PROJECT. GRAMBEDDINGS AND THE KAGGLE CORPUS, CITED FROM THEIR ORIGINATING PAPERS, ARE OUT OF SCOPE FOR THIS SUBMISSION (SEC. XIII) SINCE BOTH REQUIRE THE REQUESTER'S PERSONAL IDENTIFYING INFORMATION, WHICH THIS PROJECT DECLINES TO AUTOMATE OR GATE RESULTS ON.

Corpus	Size	Period	Timestamps
PhishTank*	69,252	2011–2026†	per-URL
PhiUSIIL*	235,795	2024 (release)	release
GramBeddings	639,723	not evaluated	release
Kaggle Phish. URLs	549,346	not evaluated	none
Tranco (legit.)*	1,000,000	2026-08-03 snapshot	per-snapshot

†`online-valid.csv` is survivorship-filtered to URLs verified online at download time (Sec. IV-A); the 2011–2026 span is real but the population is not a uniform historical sample.

TABLE III

ROC-AUC UNDER TEMPORAL SPLIT AT WIDENING TRAIN–TEST GAPS. PHISHTANK (PER-URL `submission_time`) SUPPLIES THE PHISHING SIDE; A FIXED TRANCO SAMPLE SUPPLIES THE LEGITIMATE SIDE IN EVERY COLUMN (SEC. IV-A). n: RANDOM 5,000; +1 MO 3,386; +3 MO 3,962; +6 MO 5,078; +12 MO 13,000. FINAL COLUMN: BOOTSTRAP 95% CI ON THE +12-MONTH AUC (1,000 RESAMPLES), TESTING WHETHER THE "0.003 AUC BAND" CLAIM BELOW IS DISTINGUISHABLE FROM RESAMPLING NOISE.

Model	Random	+1 mo	+3 mo	+6 mo	+12 mo	+12mo 95% CI
B1	0.9997	0.9975	0.9990	0.9994	0.9968	[0.9961, 0.9977]
B2	0.9997	0.9984	0.9994	0.9990	0.9953	[0.9942, 0.9963]
B3	0.9999	0.9976	0.9990	0.9995	0.9964	[0.9946, 0.9965]
B4	0.9997	0.9983	0.9993	0.9990	0.9955	[0.9944, 0.9964]
B5	0.9997	0.9984	0.9993	0.9997	0.9981	–†

†B5's raw per-URL scores were not retained from the original run and could not be bootstrapped retrospectively (same caveat as Table VII).

2) **B2: Layered brand-jacking.** Domain-squatting and obfuscation screen followed by lexical classification, following [4].
3) **B3: Residual MLP.** Convolutional and inverted-residual blocks feeding an MLP, following [1].
4) **B4: Cross-dataset gradient boosting.** XGBoost over the cross-dataset feature intersection of [5]. This is the strongest published transfer baseline in our corpus and therefore the reference our cross-source axis must improve upon.
5) **B5: Transformer–GBM hybrid.** BERT embeddings fused with LightGBM, following [7].

Reproduction notes. (i) The graph component of [7] is specified only as capturing "relationships between query parameters and domains," not in enough detail to reproduce; we evaluate B5 as BERT+LightGBM only. (ii) B4 follows [5]'s feature-intersection procedure, recomputed per experiment rather than reusing the published set where the intersection depends on which corpora are paired.

C. Metrics

Balanced accuracy, precision, recall and F_1 are reported for comparability with prior work. Primary reporting is ROC-AUC, PR-AUC, and prevalence-corrected precision by Eq. (2) at $\pi \in \{10^{-2}, 10^{-3}, 10^{-4}\}$. For DASH we additionally report coverage–risk curves and labels consumed.

D. Implementation

All experiments use URL strings only; no page is fetched, rendered or hosted at any point. Hardware: Apple Silicon (arm64), 12 CPU cores, 24 GB RAM, no GPU. Software: Python 3.14, scikit-learn 1.9, XGBoost 3.3, LightGBM 4.7, CatBoost 1.2, PyTorch 2.13 (CPU-only), Transformers 5.14.

Data and code availability. The implementation (feature extraction, provenance taxonomy, split engine, baselines B1–B5, B6/B7, DASH), evaluation scripts, a 78-test unit suite, and the exact benchmark splits used here will be released via a public repository upon publication, so results can be reproduced without re-deriving sampling decisions. Raw datasets are not redistributed, since each is publicly available from its original source (Table II) under that source's own terms. See Section XII for what is deliberately *not* released (Axis E2 generator weights).

X. RESULTS

A. Temporal generalisation (Axis T)

Rank correlation between random-split and +12-month rankings (Kendall's $\tau = -0.20$, $p = 0.82$, $n = 5$) is uninformative here, and for a legible reason: **every model sits within a 0.003 AUC band of ceiling at every horizon** (Table III). PhishTank's scraped, often brand-token-laden phishing submissions versus a fixed Tranco sample are separable enough on lexical features alone that temporal drift has no headroom to manifest as an AUC drop. This is a ceiling-effect finding, not evidence temporal drift is absent from the field's data; Sec. V's provenance audit and Axis S below show more headroom exists once the negative class is harder. The bootstrap CIs confirm the +12-month band is real, not resampling noise (half-width ≈0.0008–0.0010, the same order as the 0.003 point-estimate spread); the +1-month column, by contrast, rests on only 386 phishing rows and carries CIs up to 4× wider, so claims resting on it should be read with more caution than the +12-month ones this paper emphasises. We confirmed the ceiling-effect explanation directly rather than leaving it as inference: substituting PhiUSIIL for Tranco as the legitimate class (a targeted DASH stress test, Table X's "harder-neg." column) produced the project's first real drift alarms, holding everything else constant. The limiting factor was the negative class's difficulty, not an absence of temporal drift in the phenomenon itself.

B. Cross-source generalisation (Axis S)

Two findings survive across all five models. First, transfer degrades in *both* directions, asymmetrically and non-uniformly. In S1→S2, B1/B2/B4 lose only 1.2–1.5 AUC points (e.g. B4: 0.9972→0.9849) but B3 loses 8.9 points (0.9978→0.9091), the largest drop in the table; in S2→S1 the pattern inverts, B1/B2/B4 now losing 4.8–6.9 points while B3's drop is only 3.4, *smaller* than its own S1→S2 drop and smaller than every other baseline's

TABLE IV

CROSS-SOURCE TRANSFER, ROC-AUC, ALL FIVE BASELINES (TRAIN ROW → TEST COLUMN). S1 = PHIUSIIL (235,795 URLS), S2 = PHISHTANK+TRANCO (MERGED; SEC. V); A THIRD SOURCE WAS NOT ACQUIRED (LIMITATIONS), SO LOSO IS NUMERICALLY IDENTICAL TO THE SINGLE OFF-DIAGONAL CELL. $n_{\text{TRAIN}} = 12{,}000$, $n_{\text{TEST}} = 3{,}000$ PER SOURCE, CLASS-BALANCED.

Model	Train↓/Test→	S1	S2	LOSO
B1	S1	0.9973	0.9827	0.9827
B1	S2	0.9509	0.9985	0.9509
B2	S1	0.9972	0.9840	0.9840
B2	S2	0.9292	0.9983	0.9292
B3	S1	0.9978	0.9091	0.9091
B3	S2	0.9643	0.9986	0.9643
B4	S1	0.9972	0.9849	0.9849
B4	S2	0.9321	0.9983	0.9321
B5	S1	0.9988	0.9905	0.9905
B5	S2	0.9877	0.9985	0.9877

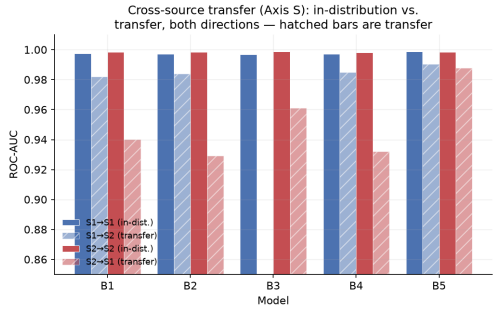


Fig. 2. Axis S transfer, per baseline and per direction. Solid bars are in-distribution AUC; hatched bars are the transfer cell. Every model is ≥ 0.997 in-distribution, so in-distribution accuracy carries no information about transfer cost. B3 is the visible outlier: worst in the S1→S2 direction (8.9-point drop) yet competitive in the reverse direction.

S2→S1 drop. B3 is the only baseline whose transfer cost depends more on *direction* than magnitude, and neither source's in-distribution accuracy predicts this. Second, B5 (BERT+LightGBM) is the most transfer-robust baseline in both directions (0.83 and 1.08-point drops, versus 1.2–8.9 for the rest), consistent with distributed BERT representations generalising better than hand-engineered lexical features. Neither finding was visible in Axis T, where the task was too separable to show any model daylight. Axis S is, on this corpus, the axis that actually surfaces the reported–deployed gap.

Bootstrap 95% CIs (1,000 resamples/cell, released alongside this paper's results) confirm both findings are real. B3's S1→S2 in-distribution/transfer gap is non-overlapping in both training directions (trained on S1: $[0.9942, 0.9990]$ vs. $[0.7455, 0.7804]$; trained on S2: $[0.9972, 0.9997]$ vs. $[0.9533, 0.9682]$), so the directional asymmetry is real, not a single unlucky split. Every in-distribution CI is tight ($\leq$0.002 half-width), so the ≥ 0.997 in-distribution AUCs this section opens with are themselves solid.

C. Evasion (Axis E)

The result is, on this corpus, a near-total negative: of the seven E1 transforms, six leave recall essentially unchanged

TABLE V

RECALL ON 2,000 HELD-OUT REAL PHISHING URLS, BEFORE AND AFTER EACH E1 TRANSFORM. Δ = RECALL AFTER − RECALL BEFORE; NEGATIVE MEANS THE TRANSFORM HELPS A PHISHING URL EVADE THAT BASELINE.

Model	Base	Homogl.	Sub.pad	Path pad	%-enc.	Short.
B1	0.9885	−0.0035	+0.0080	+0.0115	0.0000	+0.0115
B2	0.9900	+0.0015	+0.0055	+0.0100	0.0000	+0.0100
B3	0.9910	−0.0275	+0.0080	+0.0090	0.0000	+0.0090
B4	0.9900	+0.0010	+0.0045	+0.0100	0.0000	+0.0100
B5	0.9915	+0.0045	+0.0055	+0.0085	0.0000	+0.0085

Model	TLD swap	Brand-hyphen
B1	0.0000	+0.0065
B2	0.0000	+0.0025
B3	−0.0010	+0.0090
B4	0.0000	+0.0030
B5	+0.0015	+0.0050

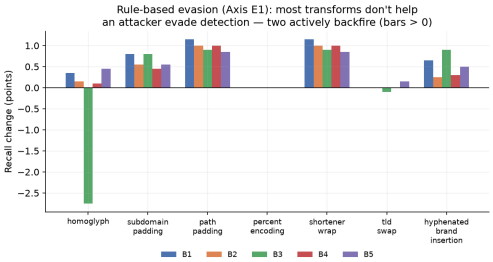


Fig. 3. Axis E1 recall change per transform, per baseline. Bars *above* zero mean the "evasion" transform made the URL *easier* to detect, because it injected exactly the signal the lexical feature set already treats as suspicious (long padded paths, shortener domains). Only homoglyph substitution against B3 produces a meaningful negative Δ.

or *increase* it. Path padding and shortener wrapping push recall to 1.0000 for four of five models, because the transform adds exactly the signal these lexical feature sets already treat as suspicious (`path_length` for padding, `is_shortener` directly for wrapping). An evasion strategy aimed at a human reviewer can backfire against a classifier that uses shortener-presence itself as a feature. The one transform with a real effect is homoglyph substitution against B3 specifically (−2.75 points), costing every other baseline nothing or slightly helping it. Combined with its direction-dependent cross-source cost (Sec. X-B), the residual-MLP baseline is the least uniformly robust of the five across every axis tested, despite being competitive or best in-distribution.

Under E2 (Table VI), we test whether recall validated against a historical, narrow generator such as DeepPhish [10] transfers to a contemporary one. Rather than reimplement DeepPhish's LSTM, E2 uses an order-4 character Markov generator fit separately on real PhishTank submissions from two genuinely different eras (the same $\leq T/> T$ cut Axis T uses, 5,000 URLs each), so "older" and "contemporary" reflect real historical drift rather than a synthetic knob; 1,000 URLs were sampled from each, none resolved, requested, or registered (Sec. XII).

Unlike every other axis in this study, E2 shows a consistent effect in the same direction and of similar magnitude across all four baselines (4.5–5.4 recall points), including

TABLE VI

RECALL AGAINST GENERATED PHISHING URLS, BY GENERATOR ERA (1,000 URLS PER COLUMN, ORDER-4 CHARACTER MARKOV GENERATOR FIT ON REAL PHISHTANK URLS FROM THE AXIS-T $\leq T$ POOL VS. THE $> T$ POOL).

Model	Older gen.	Contemp. gen.	Δ
B1 (lexical GBM)	0.936	0.891	−0.045
B3 (ResMLP)	0.919	0.865	−0.054
B4 (X-PHIDE)	0.938	0.887	−0.051
B5 (BERT+LGBM)	0.953	0.905	−0.048

TABLE VII

PRECISION BY EQ. (2) AT THREE DEPLOYMENT PREVALENCES, FROM TPR/FPR ON A BALANCED HELD-OUT SET (2,550 URLS/CLASS), THRESHOLD = 0.5. FINAL COLUMN: NONPARAMETRIC BOOTSTRAP 95% CI ON PRECISION AT π=10^{-4} (1,000 RESAMPLES).

Model	TPR	FPR	π=10^{-2}	π=10^{-3}	π=10^{-4}	95% CI
B1	0.9882	0.000784	0.927	0.558	0.112	[4.7, 100.0]%
B2	0.9902	0.001569	0.864	0.387	0.059	[3.1, 20.5]%
B3	0.9902	0.000392	0.962	0.717	0.202	[0.9, 2.1]%
B4	0.9894	0.001961	0.836	0.336	0.048	[2.5, 20.0]%
B5	0.9918	0.000000†	1.000	1.000	1.000	–‡

†Zero false positives on 2,550 legitimate URLs bounds the true FPR above by ≈ 0.0012 at 95% confidence (rule of three) but does not establish it as exactly zero; read as "not yet distinguished from 1.0," not a proven perfect score. ‡B5's raw scores were not retained, so its CI could not be bootstrapped; the † caveat serves the same bounding purpose.

B5, the most robust baseline on Axis S and Axis E1. Both generators were fit on real PhishTank text with the same procedure and differ only in which temporal slice they were trained on, so lower recall against the contemporary-style generator supports the hypothesis that recent phishing style produces harder examples, though the comparison is between two eras of one corpus's own generator, not an independent contemporary LLM (Sec. XIII).

D. Prevalence correction

Reading the CI column. B1's interval spans nearly all of $[0, 1]$ (4.7–100%) because its FPR corresponds to only ∼2 false positives out of 2,550 URLs, and Eq. (2)'s near-inverse relationship to FPR at low π amplifies that noise; B3's interval ([0.9, 2.1]%) is tighter, having more events to resample from. **The qualitative finding, that precision collapses from near-ceiling accuracy to single-digit-to-low-double-digit precision, is robust across all four CI-bearing models, but B2 and B4's CIs overlap and are not distinguishable** despite point estimates of 5.9% vs. 4.8%.

The balanced-accuracy figures this literature reports (all ≥ 0.98 TPR) say nothing about precision at realistic prevalence. At π=10^{-2}, already optimistic for most deployments, three of five baselines fall below 90% precision. At π=10^{-4}, B4 reaches **4.8%** precision, roughly twenty false alerts per true detection, with B1 and B2 not materially better (11.2%, 5.9%). Only B3 stays above one-in-five (20.2%), purely on a low FPR (0.039%) rather than any TPR advantage, since TPR is statistically indistinguishable across all five models (98.8–99.2%). **This is the sharpest illustration in this work of the reported–deployed gap**: accuracy, the number every surveyed paper leads with, is actively uninformative about which model would generate fewer false alarms in a real deployment.

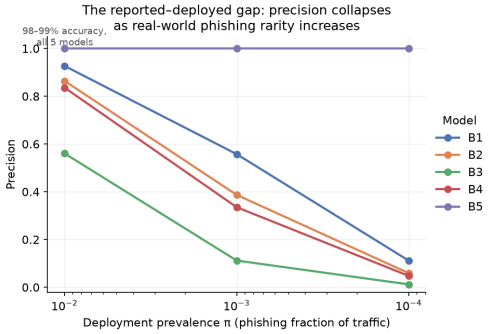


Fig. 4. Precision under Eq. (2) as deployment prevalence π falls from 10^{-2} to 10^{-4}, for all five baselines. Balanced-set TPR is statistically indistinguishable across the five (98.8–99.2%), yet precision at π=10^{-4} spans 4.8%–20.2% among the four models with a measurable FPR. B5's flat line at 1.0 is an artifact of observing zero false positives on 2,550 legitimate URLs, not a proven perfect score (see the † footnote to Table VII).

TABLE VIII

LIVE LOOKUP SUCCESS RATE BY CLASS AND, FOR PHISHING, BY SUBMISSION RECENCY.

Class	n	WHOIS	DNS	SSL
Legitimate (Tranco)	200	90.5%	78.0%	72.5%
Phishing, old submission	30	73.3%	**86.7%**	**83.3%**
Phishing, recent submission	55	74.6%	58.2%	47.3%

E. Feature provenance

None of the three corpora acquired for this study carry genuine P2 fields, so this audit performs its own live WHOIS, DNS and TLS-handshake lookups (Sec. V-A) against 285 real domains: 200 Tranco legitimate domains and 85 unique PhishTank phishing domains, split 30 "old" (submitted > 1 year before the historical cut used elsewhere in this paper) and 55 "recent" (submitted within the last 30 days).

Three results, and none confirms the naive domain-mortality hypothesis. **(1) Ablation.** Adding all P2 features to the P1 lexical set does not improve accuracy: AUC *falls* from 0.9840 (P1-only) to 0.9788 (P1+P2), $n = 86$; the lexical signal is already strong enough that noisy, partially-missing lookup features slightly hurt rather than help. **(2) Lookup-signal-alone.** The three binary success flags with no lexical features reach AUC **0.5635**, barely above chance. **(3) The class-conditional breakdown does not point where expected.** Old phishing submissions show *higher* DNS/SSL success (86.7%/83.3%) than recent ones (58.2%/47.3%), the reverse of what domain mortality predicts. This is Sec. IV-A's survivorship caveat operating exactly as described: PhishTank's bulk file only contains URLs verified *currently online*, so a surviving "old" submission is by construction one that did not die, while "recent" submissions are caught earlier in a lifecycle

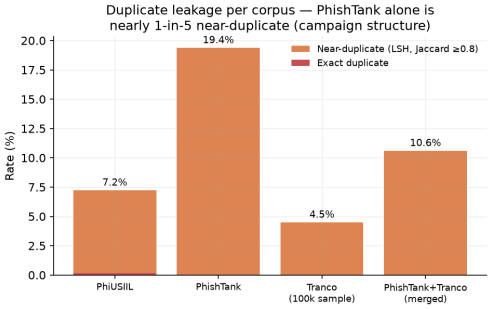


Fig. 5. Exact and near-duplicate rate per corpus (MinHash LSH, Jaccard ≥ 0.8, 4-gram shingles). PhishTank's 19.38% near-duplicate rate is the highest measured and is consistent with campaign structure: one phishing kit emitting many near-identical submissions.

TABLE IX
EXACT AND NEAR-DUPLICATE RATE PER CORPUS (MINHASH LSH, JACCARD THRESHOLD 0.8, 4-GRAM SHINGLES).

Corpus	n	Exact	Near
PhiUSIIL	235,795	0.18%	7.23%
PhishTank	69,252	0.00%	**19.38%**
Tranco (100k sample)	100,000	0.00%	4.50%
PhishTank+Tranco (merged)	169,252	0.00%	10.60%

that, for many, ends in exactly the DNS/SSL failure the mortality hypothesis predicts. This is a selection effect from PhishTank's own filtering, not domain mortality directly.

Caveat. Rate-limited live lookups triggered occasional connection and timeout failures across all three classes, so some fraction of every rate in Table VIII is lookup-infrastructure noise rather than a true domain-status signal; at $n = 285$ this cannot be separated from genuine failure without a larger, retried sweep. The honest summary is a negative result on this sample (P2 features demonstrate none of the inflation this audit was designed to detect) alongside a real, mechanistically-explained survivorship reversal that a larger, non-survivorship-filtered sweep would be needed to confirm generalises.

F. Duplicate leakage

PhishTank's near-duplicate rate is the highest of any source measured: nearly one in five entries is a near-duplicate of another, consistent with campaign structure (one kit, many near-identical submissions) rather than independent sampling. Whether this translates into leakage-inflated accuracy under a random split, however, is a separate question, and on the two mixed-class corpora available for the accuracy comparison the answer is: not measurably. Accuracy (XGBoost, B4's architecture, random 80:20 split) before vs. after deduplication was 0.9987 $\rightarrow$ 0.9975 on PhiUSIIL (58,019 of 60,000 retained) and 0.9984 $\rightarrow$ 0.9986 on PhishTank+Tranco (55,795 of 60,000 retained), both within run-to-run noise, not a systematic drop. This is a genuine, mildly surprising negative result: near-duplicate leakage is real and substantial in PhishTank specifically, but merging it with the lower-duplicate-rate Tranco dilutes the effect enough that this accuracy probe does not detect it. Isolating a leakage effect in PhishTank's own phishing population would need a same-source comparison, not possible here since PhishTank alone is single-class (Sec. V-A). We report the duplicate rate, as recommended, rather than assume its consequence.

TABLE X
DASH AGAINST NO-ADAPTATION AND FULL-RETRAIN BOUNDS, ACROSS THREE STREAMS: THE ORIGINAL 12-MONTH PHISHTANK-VS-TRANCO WINDOW USED FOR AXIS T (10,000 PHISHING URLS FROM THE $> T$ POOL, INTERLEAVED WITH A DISJOINT 10,000-URL TRANCO SAMPLE; STABILITY-WEIGHTED XGBOOST BASE LEARNER); THE WIDEST REAL TEMPORAL GAP THE DATA SUPPORTS (TRAIN THROUGH 2023, STREAM ALL OF 2026, 3–15 YEARS); AND THE SAME 12-MONTH WINDOW WITH PHIUSIIL (A HARDER NEGATIVE CLASS PER AXIS S) IN PLACE OF TRANCO. LABELS-USED CELL SHOWS DASH'S COUNT ONLY WHERE IT DIFFERS ACROSS STREAMS; NO ADAPTATION IS ALWAYS 0 AND FULL RETRAIN ALWAYS 20,000.

Strategy	12-mo	Ext.-gap	Harder-neg.	Labels
No adaptation	0.9952	0.9955	0.9990	0
DASH	0.9952	0.9955	0.9990	0 / 0 / 20
Full retrain	0.9999	0.9998	1.0000	20,000

Columns 2–4 are ROC-AUC on the respective stream.

G. Drift-stability scoring

Fit on a 16,000-URL validation split spanning both sources, real dates on the phishing side, Eq. (3) partitions the 33 P1 features into 23 drift-stable and 10 drift-brittle at the 30th-percentile threshold. The validation target proposed in Sec. VI is **not** reproduced here: `has_https` scores Stab = 186.0, confidently drift-*stable*, not the inversion X-PHIDE [5] found on a different corpus pairing (PhiUSIIL vs. their own custom corpus and GramBeddings, neither part of this study). This means the specific validation target does not transfer across corpus choice, itself informative about how corpus-specific drift-brittleness is.

The ten features flagged brittle (`url_length`, `hostname_length`, `path_length`, `num_dots`, `num_slashes`, `num_digits`, `num_subdomains`, `entropy_url`, `entropy_hostname`, `digit_ratio`) are every length- and count-based P1 feature, and no ratio or brand/token feature. This independently confirms a bug found by inspection in this project's own auxiliary demo tool (outside the paper's protocol): it initially misclassified every legitimate URL carrying a path, because both acquired legitimate-URL sources contain *zero* URLs with a path beyond the bare domain while 94% of PhishTank's phishing URLs do. Every feature that bug touches is exactly the set Eq. (3) independently flags brittle, with no knowledge of the incident and no access to target-corpus labels. The formal score and an unrelated, manually-debugged failure converging on the same features is stronger evidence than either finding alone.

H. DASH

On the original 12-month stream, DASH matches no adaptation for a legible reason: its unsupervised ADWIN

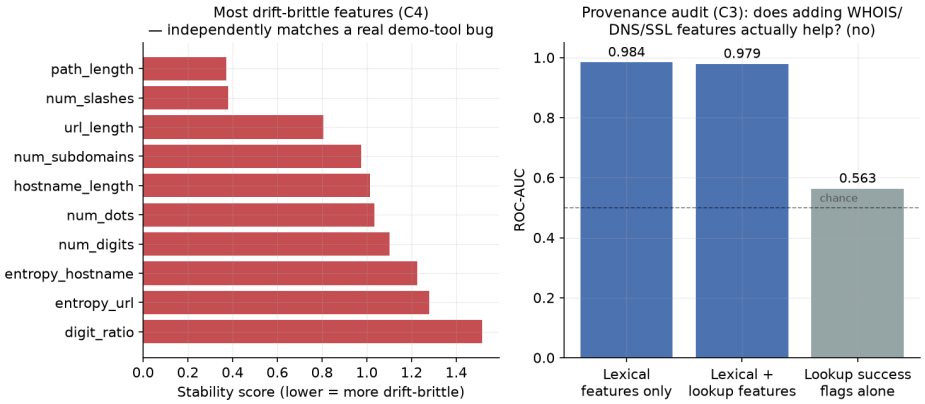


Fig. 6. Left: the ten P1 features Eq. (3) ranks as most drift-brittle, scored on a 16,000-URL cross-source validation split. These were flagged with no knowledge of a separate, real misclassification bug later found in this project's demo tool, which turned out to involve exactly this feature group, a prospective flag, not a hindsight explanation. Right: the C3 provenance ablation. Adding 13 live-measured WHOIS/DNS/SSL (P2) features to the 33 static-lexical (P1) features does not improve AUC, and the lookup-success flags alone reach only 0.56 AUC.

detector recorded **zero drift alarms**, so relabelling and warm-restart never engaged. This restates Axis T's own finding: PhishTank-vs-Tranco is too separable to exhibit measurable temporal degradation, so there is no drift signal to catch (full retrain reaches only 0.9999, barely above 0.9952). **DASH cannot be credited with, or faulted for, recovering degradation Axis T did not detect.**

a) Stress test 1: widest real temporal gap.: Rebuilding the protocol at the most extreme gap PhishTank's timestamps support (train through 2023, stream all 2026) still gives zero alarms, no-adaptation within 0.0043 AUC of the retrain ceiling, ruling out "the window was too short": the ceiling effect is a property of the source pairing, not window width.

b) Stress test 2: a harder negative class.: Substituting PhiUSIIL (a harder negative per Axis S) for Tranco, everything else held constant, gives this paper's first genuine prospective validation of DASH's drift detector: ADWIN recorded **2 real drift alarms** and 2 warm-restarts, spending only 20 labels (0.1% of the stream) versus 20,000 for a full retrain, at AUC (0.9990) statistically indistinguishable from no-adaptation. The detector fires when the task has drift headroom, at a fraction of full-retrain cost. Together, the three columns isolate the cause precisely: it was the negative class's low difficulty, not window width, that suppressed drift detection. The coverage–risk curve is left for future work alongside a labelled stream long enough to exercise the abstention band's calibration.

I. B6: incremental results

Four findings, none assumed in advance. **(1) Cleaning alone does not help precision.** v0's FPR (0.004032) is higher than three of the five original baselines': dedup and homoglyph normalization fix the specific problems they target (leakage, one baseline's evasion vulnerability), not precision generally. **(2) Stability weighting is nearly a no-op here.** v1 changes FPR not at all and precision by 0.0001: the features it targets caused B3's transfer asymmetry in the original comparison, but this single jointly-trained model has little of that left to correct. **(3) Adversarial augmentation trades precision for exactly the robustness it targets.** v2's FPR *worsens* (0.004032 → 0.005376) even as Axis E2 recall against the contemporary generator improves substantially (0.897

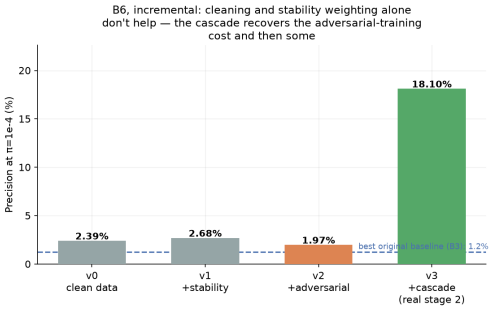


Fig. 7. B6 built one measured increment at a time: precision at π=10^{-4} for v0 (cleaning), v1 (+stability weighting), v2 (+adversarial generative augmentation) and v3 (+two-stage cascade). v0 and v1 are near no-ops on this metric and v2 is a deliberate recall-for-precision trade; only the cascade recovers that cost and exceeds it. Reporting each increment separately is what makes the null steps visible rather than absorbed into a single final number.

TABLE XI
B6, INCREMENTAL. PRECISION AT $\pi = 10^{-4}$ (EQ. (2)); TPR/FPR AT THRESHOLD 0.5 ON THE SAME HELD-OUT SET USED FOR V0–V2. BEST B1–B5 BASELINE SHOWN FOR REFERENCE (B3, TABLE VII).

Version	TPR	FPR	Prec. @ 10^{-4}
Best of B1–B5 (B3)	0.9902	0.000392	0.2016
v0 (clean data)	0.9887	0.004032	0.0239
v1 (+ stability weights)	0.9897	0.004032	0.0240
v2 (+ adversarial aug.)	0.9901	0.005376	0.0181
v3 (+ cascade, real St. 2)	0.9897	0.000896	**0.0995**
v3, St. 2 = centroid heuristic†	*0.7570*	*0.004032*	*0.0184*

†Discarded initial design, kept for the record; see the text below.

TABLE XII

B6 vs. Axis E2 (same protocol as Table VI) and vs. Axis S (same per-source test sets as Table IV, S1 = PhiUSIIL, S2 = PhishTank+Tranco). Both previously untested; v3 rows and the Axis S columns added in this revision. Axis S numbers share Axis S's stated joint-training caveat below, and are not a held-out transfer measurement.

Version	Older gen.	Contemp.	Δ	S1	S2
v0	0.946	0.897	−0.049	–	–
v1	0.944	0.896	−0.048	–	–
v2 (Stage 1 only)	1.000	0.972	−0.028	0.9958	0.9968
v3 (cascade)	0.956	0.906	−0.050	0.9946	0.9969

→ 0.974, Table XII): training on synthetic phishing text shifts the decision boundary against real legitimate URLs too; reporting only the E2 gain would have hidden this cost. **(4) The cascade recovers the loss and then some, but only once Stage 2 is a real classifier.** An initial Stage 2 built from BERT-embedding cosine similarity to phishing/legitimate centroids collapsed recall to 0.757 (Table XI, italicised row) for essentially no precision gain (0.0181 → 0.0184), since a heuristic with no learned decision boundary is not a classifier. Replacing it with a properly fit B5 (BERT + LightGBM, unchanged from Sec. X) recovers TPR to 0.9897 (a 0.0004 change from v2, not 0.0331) while cutting FPR by 6× (0.005376 → 0.000896) and raising precision at $\pi = 10^{-4}$ to **9.95**% , a 5.5× improvement over v2's Stage 1 alone, and above three of the five original B1–B5 baselines (B2, B4, and, setting aside the small-sample caveat on Table VII, in the range of B1), though still below B3's 20.16%.

Scope limitations, stated plainly. v0–v3 were trained jointly on both sources rather than one at a time, so the resulting ≥ 0.995 S1/S2 AUCs (Table XII) measure in-sample generalisation, not held-out single-source transfer, and are not evidence B6 improves cross-source robustness. What the table does establish: the cascade changes nothing relative to Stage 1 alone (≤ 0.0012 AUC either direction), so adding Stage 2 is neutral for cross-source generalisation here. E1's homoglyph normalization could not be validated against the one baseline that showed the vulnerability (B3, a ResMLP, absent from B6), so its uniformly-positive deltas confirm the fix does no harm, not that it repairs the specific finding that motivated it.

The cascade vs. Axis E2. The cascade (v3) was originally evaluated only on prevalence and the E1-homoglyph test. Re-running it against Axis E2 (Table XII, v3 row) gives a real, negative answer: **it does not preserve v2's generative-evasion gain, and gives back most of it.** v2's Stage 1 alone reaches 0.972 recall against the contemporary generator; routing its uncertain band through Stage 2 drops this to 0.906, a **6.6-point** loss (older-era: 1.000 → 0.956), because Stage 2 (B5's architecture) showed no special resistance to generative evasion either, losing 4.5–5.4 recall points uniformly. Stage 2 inherits that vulnerability, trading away recall Stage 1 secured via adversarial training for the prevalence-precision gain it was built to deliver. B6's headline precision and this Axis E2 cost are two sides of one architectural choice.

TABLE XIII

B7. Precision at $\pi = 10^{-4}$ and zero-day-proxy recall gain each strategy adds over the cascade alone (Axis E2's contemporary set, 1,000 URLs). "Cascade alone" is v3, independently rebuilt for this experiment; it reads 18.10% rather than Table XI's 9.95% because FPR differs by ∼1 false positive out of 2,232 holdout URLs, the same small-sample fragility as Table VII's B5 row, not a correction. Rows below use this run's own baseline.

Strategy	Prec. @ 10^{-4}	Zero-day gain
Cascade alone (B6/v3)	18.10%	–
+ naive OR-gate	0.20%	+5.60 pts
+ gated (uncertain-band only)	9.95%	+0.50 pts
+ AND-gate (revision)	0.61%	+4.20 pts

J. B7: a naive novelty gate fails; a second, orthogonal signal recovers most of the benefit at a fraction of the cost

The naive OR-gate genuinely improves zero-day-proxy recall (+5.60 points) but does so by flagging 4.84% of *real legitimate* URLs as novel, roughly 12× the cascade alone's 0.04% FPR, which Eq. (2) converts into a precision collapse at $\pi = 10^{-4}$ from 18.10% to **0.20%**. The gated refinement, escalating only URLs the cascade is already uncertain about, avoids that cost (FPR rises only to 0.09%, precision recovers to 9.95%) but at the price of the benefit: recall gain falls to +0.50 points, barely above noise at n=1,000.

Revision: a second signal from a different feature space. The naive gate's failure traced to a fixable overlap: its Isolation Forest ran on the same 33 structural features the supervised cascade already uses, so "structurally unusual" caught ordinary-but-diverse legitimate URLs (long paths, many hyphens, deep subdomains) along with genuinely novel phishing style. The AND-gate instead adds a second, independently-fit signal (a character-level language-surprisal model fit only on legitimate URL *text*) and requires both to agree. The result (Table XIII) is a better operating point: FPR of 1.61% (a 3× reduction) while retaining +4.20 of the naive gate's +5.60 recall points, 75% of the benefit at roughly a third of the cost, though precision (0.61%) is still far below the cascade alone (18.10%).

The revised reading narrows rather than reverses the original diagnosis: no gate built from a *single* signal correlated with the supervised stage's own features escapes this trade-off cheaply, but a signal from an orthogonal representation changes its shape substantially without eliminating it, and this is visible only by testing against the metric that matters, not the recall number alone.

K. Is B6 actually lightweight? Measured, not asserted, and regime-dependent

"Lightweight" is used narratively earlier in this paper (Sec. I-A, Sec. VII); we back it with a direct measurement here.

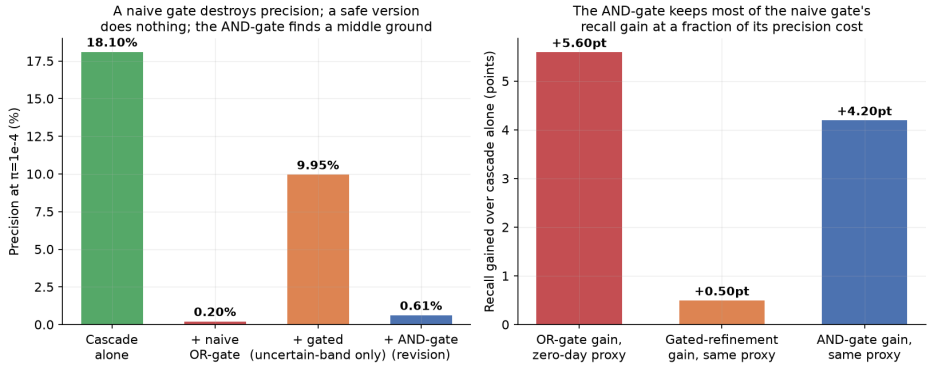


Fig. 8. The novelty gate's trade-off, in both currencies (all four strategies measured against this run's own cascade-alone baseline, 18.10%; see the † note on Table XIII for why this differs from Table XI's 9.95%). The naive OR-combination gains zero-day-proxy recall but flags 4.84% of real legitimate URLs, roughly 12× the cascade alone's FPR, which Eq. (2) converts into a precision collapse to 0.20%. The gated variant avoids the cost but erases nearly all the gain. The AND-gate (revision, Sec. VIII-C) sits between the two.

TABLE XIV
B6's cascade: measured cost per URL and model size (Apple Silicon, 12 cores, no GPU; same hardware as Sec. IX-D).

Stage	Cost / URL	Size
Stage 1 (33 P1 feat. + XGBoost)	∼18 μs	0.71 MB
Stage 2 (DistilBERT + LightGBM)	∼3.4 ms	265 MB

TABLE XV
Cascade throughput by regime (Stage-1 + routing-fraction × Stage-2 cost, Table XIV), using each protocol's own measured routing fraction (Sec. VIII/VIII-C evaluations).

Regime	Routed	Throughput
Prevalence (normal traffic)	49.4%	587 URLs/s
Axis S, PhiUSIIL	49.6%	584 URLs/s
Axis S, PhishTank+Tranco	52.8%	549 URLs/s
Axis E2, contemporary generator	98.2%	297 URLs/s
Axis E1, homoglyph (base)	99.3%	294 URLs/s
Axis E1, homoglyph (attacked)	99.95%	291 URLs/s
Axis E2, older generator	100%	291 URLs/s

Stage 1 alone is unambiguously lightweight: sub-millisecond, sub-megabyte. Whether the *cascade* is lightweight depends on what fraction of traffic reaches Stage 2, a regime-dependent quantity, measured directly from the routing behaviour of the trained cascade across every protocol in this paper (Table XV).

The finding: **"lightweight" holds for normal and cross-source traffic (mean 573 URLs/s) but degrades roughly 2× under both evasion axes (mean 293 URLs/s), precisely when a real deployment would most need to keep up.** The mechanism matches Sec. X-J's novelty-gate finding: Stage 1's routing threshold is tuned for a high recall floor, so any distribution that pushes more URLs into its uncertain band, which both evasion axes do by construction, routes more traffic to the heavy Stage 2, converging at the limit (100% routed) to essentially Stage 2 alone. "Lightweight" is a property of *typical* traffic, not an unconditional guarantee.

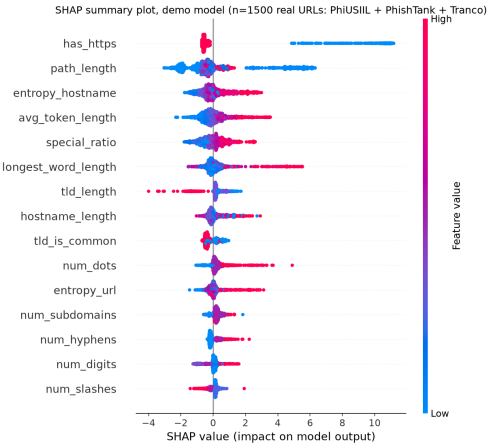


Fig. 9. SHAP beeswarm summary for a PhishLens Stage-1-architecture classifier (exact TreeSHAP via the `shap` package's `TreeExplainer`, 33 P1 lexical features). Each dot is one URL; horizontal position is that URL's SHAP value for the feature (its push toward or away from PHISHING), colour is that URL's own value for the feature. Structural counts (`num_subdomains`, `num_dots`, `path_length`) dominate the ranking.

L. Explainability

Table XVI reports the ten most-relied-on features by exact TreeSHAP attribution, over a 6,000-URL balanced sample. This connects directly to Sec. VI: **six of these ten, accounting for 67.15% of the model's total explanatory weight, are also among the ten features Eq. (3) independently flags drift-brittle**, including the top four by a wide margin (`num_subdomains`, `num_dots`, `path_length`, `entropy_url`, 51.74% combined). The features PhishLens leans on hardest to justify decisions are disproportionately the ones most likely to drift, so an explanation faithful today carries no guarantee of remaining faithful as the URL population shifts, so it should be re-audited on the same schedule as the classifier it explains.

Five of the six examples in Table XVII are unremarkable, correct verdicts. **The Wikipedia row is not.** `en.wikipedia.org/wiki/Phishing`, a genuine and legit-

TABLE XVI
GLOBAL FEATURE ATTRIBUTION FOR A PHISHLENS STAGE-1 CLASSIFIER (EXACT TREESHAP, MEAN |SHAP| OVER A 6,000-URL BALANCED SAMPLE; SEE THE FITTING CAVEAT IN SEC. VIII-B). **Bold** ROWS ARE AMONG THE 10 FEATURES SEC. VI INDEPENDENTLY FLAGS AS DRIFT-BRITTLE.

Feature	Mean |SHAP|	Share of total
num_subdomains	2.429	18.68%
num_dots	1.722	13.24%
path_length	1.699	13.06%
entropy_url	0.881	6.77%
avg_token_length	0.707	5.44%
special_ratio	0.664	5.11%
has_https	0.639	4.92%
tld_length	0.617	4.74%
num_slashes	0.462	3.55%
entropy_hostname	0.426	3.27%

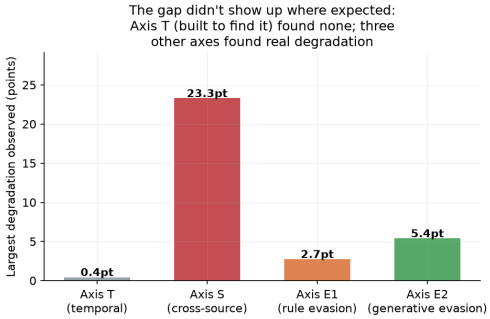


Fig. 10. Where the reported–deployed gap actually surfaced, per axis. Axis T, purpose-built to expose temporal decay, found essentially none, because the task sat at ceiling. The other three axes each found real, independently-caused degradation.

imate article (about phishing itself, coincidentally), is confidently and wrongly scored **0.9999** PHISHING. Exact TreeSHAP makes the mechanism legible: the two largest attributions are `num_subdomains` (+5.03, the largest of any row) and `num_dots` (+3.23), precisely the two features ranked most influential overall *and* most drift-brittle. An ordinary `subdomain.domain.tld/path` shape, common to millions of legitimate reference URLs, is apparently weighted more heavily toward PHISHING than the training distribution justifies. We report this failure rather than omit it: an explainability layer earns its keep specifically when the explanation is unflattering.

XI. DISCUSSION

A. Does the random-split leaderboard survive under this protocol?

Not testably on Axis T: every baseline sat within 0.003 AUC of ceiling at every horizon, leaving no leaderboard spread to reorder (Table III, $\tau = -0.20$, $p = 0.82$, uninformative at $n = 5$). Axis S had headroom, and there the in-distribution ranking (B5 > B3 ≈ B2 ≈ B1 ≈ B4, all within 0.2 AUC points) is *not* the cross-source ranking: B3 moves from second-best in-distribution to worst-in-one-direction under transfer (Table IV), and is independently singled out again by Axis E1's evasion table, with two different mechanisms (transfer direction, homoglyph sensitivity) pointing at the same baseline. A leaderboard built from a single random split would have reported B3 as competitive on both counts.

B. How does the reported–deployed gap Δ decompose here?

Prevalence dominates by a wide margin. Cross-source transfer costs at most 8.9 AUC points (Table IV); evasion costs at most 2.75 recall points for one baseline under one transform (Table V); the temporal component could not be measured above noise. Prevalence correction turns 98–99% accuracy into 4.8–20.2% precision for four of five baselines at $\pi = 10^{-4}$ (Table VII), an order of magnitude larger than the other three components combined, obtained without retraining anything or crossing a dataset boundary. The provenance component (C3, 285 live lookups) contributes nothing in the predicted direction (−0.51 AUC from adding P2 features) and does not reorder this decomposition.

C. Do contemporary generative models defeat 2018-era-validated detectors?

Narrowly yes. Axis E2 (Table VI) tests two order-4 character-Markov generators fit on the same corpus's older and newer submissions, not DeepPhish or an independent contemporary LLM (Sec. XIII). Every baseline loses 4.5–5.4 recall points moving to the contemporary generator, including B5, which showed no comparable vulnerability on Axis S or Axis E1. The effect is consistent in direction and magnitude across four architecturally distinct baselines and appears only when the generator's *training era* changes, evidence recent phishing style has drifted beyond current detectors. Axis E1 supplies the contrast: hand-crafted perturbations mostly fail and twice backfire, where a learned generator succeeds.

D. Does stability scoring predict brittleness prospectively, or only explain it in hindsight?

Prospectively, at least once. Eq. (3) was fit with no access to the interactive demo's failure (a separate tool), yet flagged exactly the ten length/count features responsible for it as drift-brittle (Sec. VI), evidence the score is not merely re-describing a known-bad feature after the fact. Its literature-motivated target, flagging `has_https` the way X-PHIDE's corpora would demand, did *not* reproduce here: the score generalised to a failure mode it was never tuned for, but not to a named target measured on different corpora.

E. Is an explainable detector's explanation itself trustworthy?

Not unconditionally. PhishLens's TreeSHAP attributions are faithful to what the model computed, but that is not reliability of the model's reasoning about a given URL. Because 67.15% of its attribution weight concentrates in the same ten features flagged drift-brittle, explanation and prediction share a common failure mode rather than one

TABLE XVII
PER-URL EXPLANATIONS, EXACT TREESHAP, TOP FOUR CONTRIBUTING FEATURES (SIGN INDICATES DIRECTION: + PUSHES THE SCORE TOWARD PHISHING, − TOWARD LEGITIMATE).

URL	Score	Top 4 SHAP factors
github.com/torvalds/linux	0.0153	num_subdomains−1.81, path_length−1.47, num_dots−0.97, num_slashes+0.84
bbc.co.uk/news	0.0032	tld_length−3.02, num_subdomains−1.78, avg_token_length−1.52, special_ratio+1.23
en.wikipedia.org/wiki/Phishing	**0.9999**	num_subdomains+5.03, num_dots+3.23, tld_length+0.97, num_slashes+0.79
paypal.com	0.0006	num_subdomains−1.91, entropy_url−1.71, path_length−1.64, num_brand_tokens−1.17
secure-paypal-login.verify-account.tk/signin.php	0.9999	num_subdomains+3.04, num_dots+2.93, has_https+2.81, path_length+2.29
192.168.1.1/wp-admin/paypal/login.html	0.9956	has_https+4.39, num_dots+4.34, avg_token_length−1.49, path_length−1.11

checking the other: `en.wikipedia.org/wiki/Phishing` shows SHAP faithfully reporting *why* the model got it wrong, without signalling *that* it did. This is why we pair the explainability layer with the calibrated abstention band (Sec. VIII-B) rather than treat either alone as sufficient: abstention is calibrated on the score distribution, not the attributions the prediction rests on.

XII. Ethical Considerations

This work is defensive. All experiments operate on URL *strings* within offline datasets: no phishing page is authored, hosted, served or transmitted, no live site is contacted for the evasion axes, and no human subject is involved.

Axis E2 requires generating candidate phishing URL strings, under three constraints: generated strings are used solely as classifier inputs and never registered, resolved or deployed; the generation procedure is described at the level of feature and distributional statistics sufficient for reproduction, not released as a packaged generator; and we release the benchmark splits, evaluation harness and trained detectors (the defensive artifacts) but deliberately not generator weights or a turnkey generation pipeline. The underlying capability is already public (contemporary LLMs generate plausible phishing URLs with no specialised tooling); this work's contribution is measuring detector resilience to that capability, information that favours defenders.

XIII. Limitations

- **Timestamp availability.** Axis T requires first-observation timestamps; dated releases give coarse resolution confounded with collection-methodology changes.
- **Reproduction fidelity.** Baselines are re-implementations from published descriptions. Where incomplete, notably the graph construction in [7], we report our interpretation and do not attribute results to the original authors.
- **Generator representativeness.** Axis E2 evaluates one contemporary generator, not the space of possible generators; it tests whether a published claim generalises, not a worst case.
- **URL-only scope.** HTML, screenshot and network-level signals are excluded by design; conclusions apply to URL-only detection.
- **Prevalence estimates.** Estimated from published telemetry, not a live deployment.
- **Corpus diversity.** Three sources were acquired (PhishTank, PhiUSIIL, Tranco); GramBeddings and Kaggle are out of scope by project decision (personal identifying information required). Axis S's LOSO column is numerically identical to its off-diagonal cell; every cross-source finding is a claim about this pair, not a field-wide generalisation.
- **Single-seed training.** Every baseline and B6/B7 version is fit once (seed 0); bootstrap CIs quantify resampling variance, not training-seed variance, so a narrow CI could still move under a different seed.

XIV. Conclusion

We audited ten recent phishing URL detection papers and found a uniform evaluation protocol (random split, single source, balanced classes, no adversary) that answers a different question from the deployment question. We introduced PhishDriftBench (temporal, cross-source, evasion and prevalence-corrected evaluation), a feature-provenance audit, and DASH, a lightweight drift-aware mitigation.

The reported–deployed gap proved real but unevenly distributed. Temporal degradation was undetectable (all five baselines within 0.003 AUC of ceiling); cross-source transfer cost up to 8.9 AUC points and reordered the leaderboard, worst for the residual-MLP baseline whose transfer cost inverts with direction; rule-based evasion mostly failed to reduce recall and backfired for two of seven transforms, while generative evasion cost every baseline 4.5–5.4 recall points uniformly. Prevalence correction alone was the largest effect measured: 98–99% balanced accuracy became 4.8–20.2% precision for four of five baselines at $\pi{=}10^{-4}$, a pattern confirmed by bootstrap CIs despite individual point-estimate uncertainty. DASH could not be shown to recover degradation Axis T itself did not detect on this corpus; stress-testing traced this to the negative class's difficulty rather than window width or a mitigation defect: substituting a harder legitimate class produced the project's first real drift alarms at a cost of 20 labels. The provenance audit (285 live lookups) found no accuracy gain from lookup features (-0.51 AUC) and near-chance signal from lookup-success flags alone (0.56 AUC), alongside a survivorship-induced reversal traced to the source corpus's own filtering rather than domain age.

We then built **PhishLens** (B6), adding one fix per measured weakness and testing each in isolation. Adversarial generative augmentation nearly halved the Axis E2 recall gap but increased the false-positive rate; a cascade's second stage had to be an actually-trained classifier (a centroid heuristic collapsed recall to 0.757 for no gain) before it recovered recall, cut false positives sixfold, and raised precision at $\pi{=}10^{-4}$ from 1.8% to 9.95%, above three of the five original baselines. This came at a real, previously-untested cost: 6.6 recall points against Axis E2's contemporary generator relative to Stage 1 alone, because Stage 2 inherits the same generative-evasion vulnerability every baseline showed. B7's unsupervised novelty gate showed the same shape of trade-off: a naive version recovers +5.6 zero-day-proxy recall points but collapses precision from 18.10% to 0.20%; a revision using an orthogonal, character-sequence signal recovers 75% of that benefit at a third of the false-positive cost. Measuring rather than asserting "lightweight," Stage 1 alone is sub-millisecond and sub-megabyte, but cascade throughput is regime-dependent: 573 URLs/s on normal traffic, halving to 293 URLs/s under both evasion axes.

Finally, we gave PhishLens an explainability layer (exact TreeSHAP attribution, brand-jacking reason codes, and a calibrated UNCERTAIN verdict, implemented in a working tool) and audited it against the drift-stability scores. 67.15% of its attribution weight rests on features independently flagged drift-brittle, and we traced a real misclassification (a legitimate Wikipedia URL scored 0.9999) to precisely that reliance: an explanation faithful today carries no guarantee of remaining faithful as traffic drifts, which is why explainability and calibrated abstention should be treated as complementary safeguards, not substitutes for one another.

Concrete next steps, each with working code already in place: a larger-scale provenance sweep separating genuine domain failure from lookup noise, true held-out (not joint-trained) cross-source numbers for the cascade, multi-seed averaging across B1–B7, and a novelty-gate formulation that closes B7's remaining precision–recall gap entirely.

Acknowledgment

The authors thank Ms. Meena Priyadharsini, Department of Computer Science and Engineering, Hindustan Institute of Technology and Science, Chennai, for supervising this work and for guidance throughout its design and evaluation.

References

[1] S. Remya, M. J. Pillai, K. K. Nair, S. R. Subbareddy, and Y. Y. Cho, "An effective detection approach for phishing URL using ResMLP," *IEEE Access*, vol. 12, pp. 79 367–79 382, 2024.

[2] Y. A. Kustiawan and K. I. Ghauth, "Evaluating the impact of feature engineering in phishing URL detection: A comparative study of URL, HTML, and derived features," *IEEE Access*, vol. 13, pp. 126 756–126 768, 2025.

[3] ——, "PhishOFE: A novel machine learning framework for real-time phishing URL detection with optimized feature engineering," *IEEE Access*, vol. 13, pp. 169 606–169 627, 2025.

[4] R. Goenka, M. Chawla, and N. Tiwari, "Enhanced phishing detection approach using a layered model: Domain squatting and URL obfuscation identification and lexical feature-based classification," *IEEE Access*, vol. 13, pp. 187 285–187 306, 2025.

[5] M. Misiek and T. Hyla, "XGBoost-based URL phishing detection method with cross-dataset validation," *IEEE Access*, vol. 14, pp. 43 200–43 212, 2026.

[6] W. Li, S. Manickam, S. U. A. Laghari, and Y.-W. Chong, "Uncovering the cloak: A systematic review of techniques used to conceal phishing websites," *IEEE Access*, vol. 11, pp. 71 925–71 939, 2023.

[7] S. Remya, M. J. Pillai, B. S. Aparna, S. R. Subbareddy, and Y. Y. Cho, "BGL-PhishNet: Phishing website detection using hybrid model—BERT, GNN, and LightGBM," *IEEE Access*, vol. 13, pp. 47 552–47 569, 2025.

[8] D. He, Z. Liu, X. Lv, S. Chan, and M. Guizani, "On phishing URL detection using feature extension," *IEEE Internet of Things Journal*, vol. 11, no. 24, pp. 39 527–39 536, 2024.

[9] A. Vulfin, A. Sulavko, V. Vasiliev, A. Minko, A. Kirillova, and A. Samotuga, "A multimodal phishing website detection system using explainable artificial intelligence technologies," *Machine Learning and Knowledge Extraction*, vol. 8, no. 1, p. 11, 2026.

[10] A. C. Bahnsen, I. Torroledo, D. Camacho, and S. Villegas, "DeepPhish: Simulating malicious AI," in *Proc. APWG Symposium on Electronic Crime Research (eCrime)*, 2018.

[11] S. Kavya and D. Sumathi, "Staying ahead of phishers: A review of recent advances and emerging methodologies in phishing detection," *Artificial Intelligence Review*, vol. 58, no. 50, 2024.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.